

Unit 1

Register Transfer and Microoperations

- ☐ Register Transfer Language,
- ☐ Register Transfer,
- ☐ Bus and Memory Transfers,
- ☐ Arithmetic Microoperations,
- ☐ Logic Microoperations,
- ☐ Shift Microoperations,
- ☐ Arithmetic Logic Shift Unit

Curated By: Dr. Rohit Mathur

- **Computer Architecture: (Ref: M. M. Mano)**
 - Introduction, computer types, functional units, Von Neumann and Harvard architectures, RISC and CISC Architectures.
 - Register Transfer and Microoperations, microprogrammed control Design of Basic Computer, Design of Accumulator Logic, Control unit design, Parallel Processing, Pipelining, Input-Output Organization, Memory Organization.
- **8086 Microprocessor: (Ref: K. M . Bhurchandi & A. K. Ray)**
 - 8086 Architecture, 8086 Instruction Set: types of instructions and addressing modes, assembler and assembler directives, assembly language programming.
- **RISC V: (Ref: D. A. Patterson, and J. L. Hennessy)**
 - Introduction to the Reduced Instruction Set Computer, RISC V architecture and features.

- By the end of this course, students will be able to:
- 1) **Demonstrate** understanding of Register Transfer and Microoperations.
 - 2) **Demonstrate** understanding of control unit and its subsystem design.
 - 3) **Explain** concepts and impact of cache memory, pipelining and parallel processing on system performance.
 - 4) **Explain** the operation of various addressing modes of 8086 microprocessor.
 - 5) **Develop** Logical and Conditional assembly language programming skills for 8086 microprocessor.

- 1) M. Morris Mano, “Computer System Architecture”, Pearson, (3e), 2007.**
- 2) K. M . Bhurchandi, A K Ray, Advanced Microprocessors and Peripheral Devices”, McGraw Hill, (3e), 2018.**
- 3) David A. Patterson, and John L. Hennessy, Computer Organization and Design, The Hardware / Software Interface: RISC-V Edition, Morgan Kaufmann, 2018.**
- 4) Sonal Yadav, Computer System Organization, 1 (e), All India Council for Technical Education, 2024.**
- 5) V.C. Hamacher, Z. Vranesic & S. Zaky, “Computer Organization”, McGraw Hill, (5e), 2002.**

Computer Architecture and Computer Organization

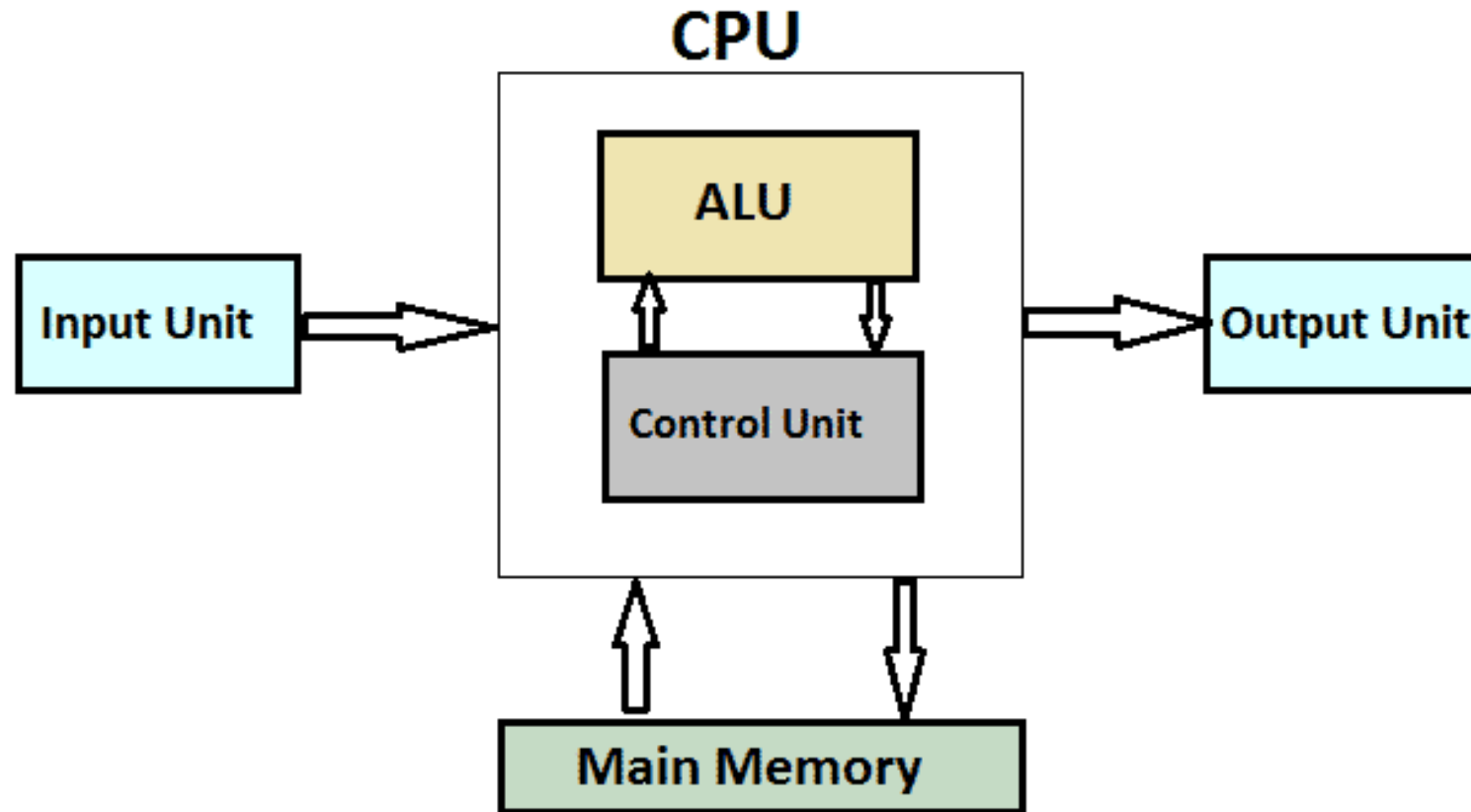
Computer Architecture:

- ✓ Computer Architecture is a functional description of requirements and design implementation for the various parts of computer.
- ✓ It deals with functional behavior of computer system.
- ✓ It comes before the computer organization while designing a computer.
- ✓ Architecture describes what the computer does.

Computer Organization:

- ✓ Computer Organization comes after the decide of Computer Architecture first.
- ✓ Computer Organization is how operational attribute are linked together and contribute to realize the architectural specification.
- ✓ Computer Organization deals with structural relationship.
- ✓ Organization describes how it does it.

Srno.	Computer Architecture	Computer Organization
1	Architecture describes what the computer does.	Organization describes how it does it.
2	Computer Architecture deals with functional behavior of computer system.	Computer Organization deals with structural relationship.
3	it deals with high-level design issue.	it deals with low-level design issue.
4	Architecture indicates its hardware.	Where, Organization indicates its performance.
5	Computer Architecture is also called as instruction set architecture.	Computer Organization is frequently called as micro architecture.
6	Computer Architecture comprises logical functions such as instruction sets, registers, data types and addressing modes.	Computer Organization consists of physical units like circuit designs, peripherals and adders.



A computer consists of 5 functionally independent main parts:

- 1)Input,
- 2)Memory,
- 3)Arithmetic & logic,
- 4)Output and
- 5)Control units.

Input Unit • The computer accepts the information in the form of program & data through an input device. Eg: keyboard

Whenever a key is pressed, the corresponding letter/digit is automatically translated into its corresponding binary-code and transmitted over a cable to either the memory or the processor.

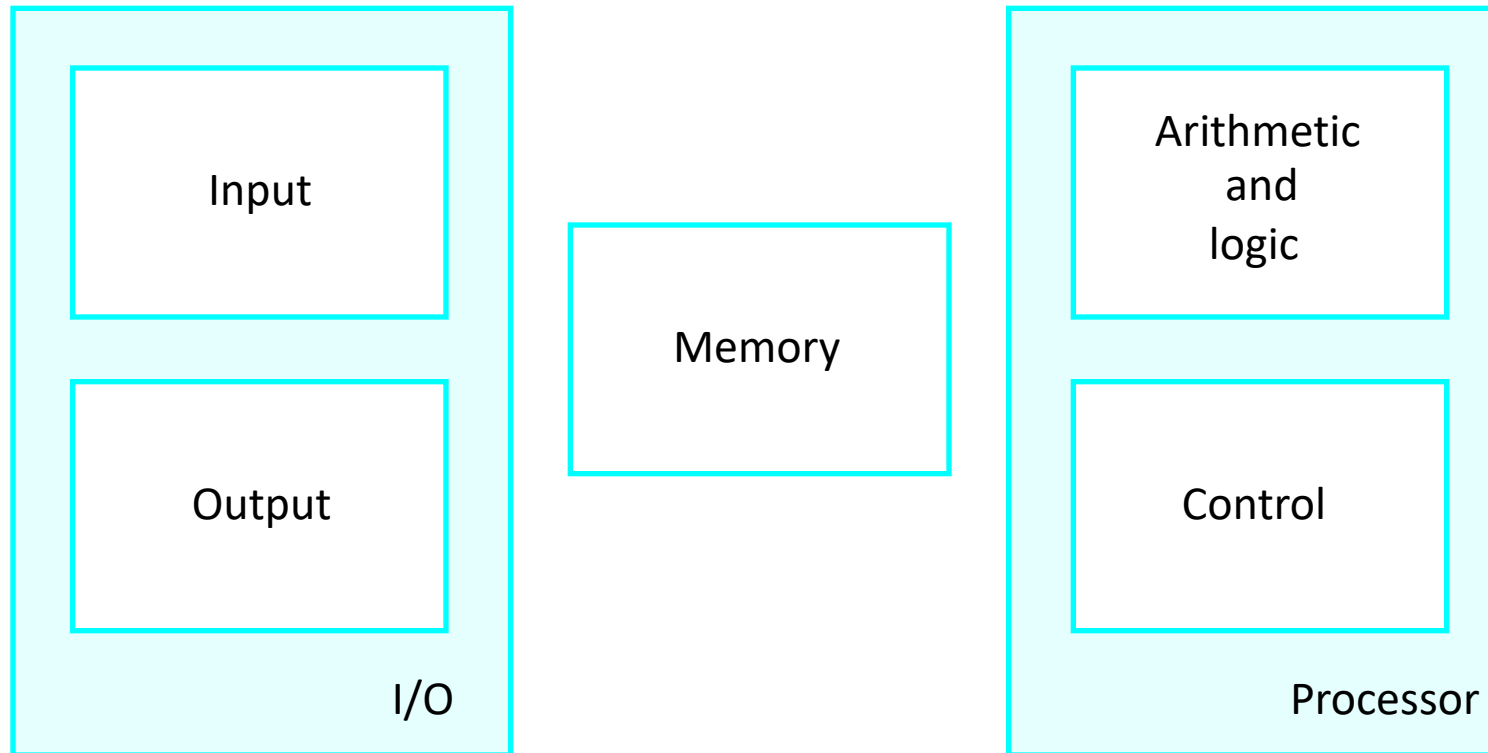
Memory Unit • This unit is used to store programs & data. • There are 2 classes of storage: 1) Primary-storage is a fast-memory that operates at electronic-speed. Programs must be stored in the memory while they are being executed. 2) Secondary-storage is used when large amounts of data & many programs have to be stored. Eg: magnetic disks and optical disks(CD-ROMs). • The memory contains a large number of semiconductor storage cells(i.e. flip-flops), each capable of storing one bit of information. • The memory is organized so that the contents of one word can be stored or retrieved in one basic operation.

Memory in which any location can be reached in a short and fixed amount of time after specifying its address is called RAM (Random Access Memory).

ALU (Arithmetic & Logic Unit) • This unit is used for performing arithmetic & logical operations. • Any arithmetic operation is initiated by bringing the required operand into the processor (i.e. registers), where the operation is performed by the ALU.

Output Unit • This unit is used to send processed-results to the outside world. Eg: printer, graphic displays etc.

Control Unit • This unit is used for controlling the activities of the other units (such as memory, I/O device). • This unit sends control-signals (read/write) to other units and senses their states. • Data transfers between processor and memory are also controlled by the control-unit through timing-signals. • Timing-signals are signals that determine when a given action is to take place.



Basic functional units of a computer.

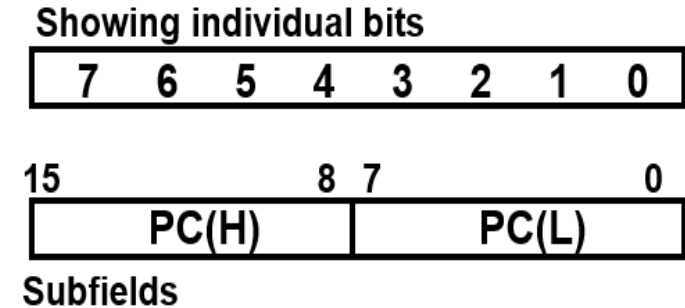
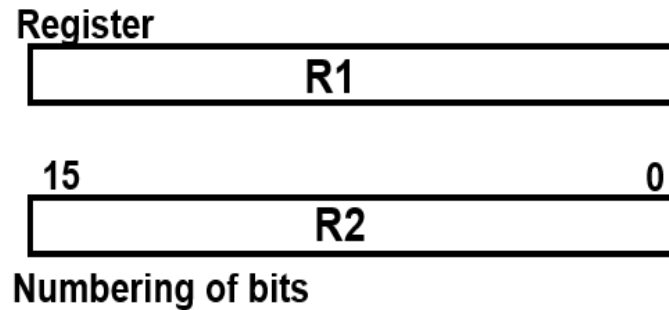
- Combinational and sequential circuits can be used to create simple digital systems.
- These are the low-level building blocks of a digital computer.
- Typically,
 - What operations are performed on the data in the registers
 - What information is passed between registers
- The operations on the data in registers are called microoperations.
- The functions built into registers are examples of microoperations.
- Eg: Shift, Load, Clear, Increment, etc.

- Definition of the (internal) organization of a computer
 - Set of registers and their functions
 - Microoperations set: Set of allowable microoperations provided by the organization of the computer
 - Control signals that initiate the sequence of microoperations (to perform the functions)

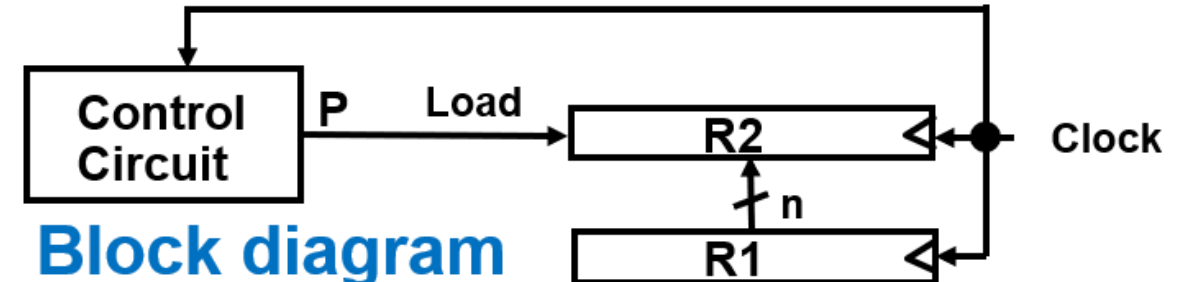
- Rather than specifying a digital system in words, a specific notation is used, *register transfer language*
- For any function of the computer, the register transfer language can be used to describe the (sequence of) microoperations
- Register transfer language
 - A symbolic language
 - A convenient tool for describing the internal organization of digital computers
 - Can also be used to facilitate the design process of digital systems.

- Designation of a register:

- a register
- portion of a register
- a bit of a register



- Copying the contents of one register to another is a register transfer
 - A register transfer is indicated as
 - $R2 \leftarrow R1$
- Often actions need to only occur if a certain condition is true
- In digital systems, this is often done via a control signal, called a control function
 - If the signal is 1, the action takes place
- This is represented as: $P: R2 \leftarrow R1$



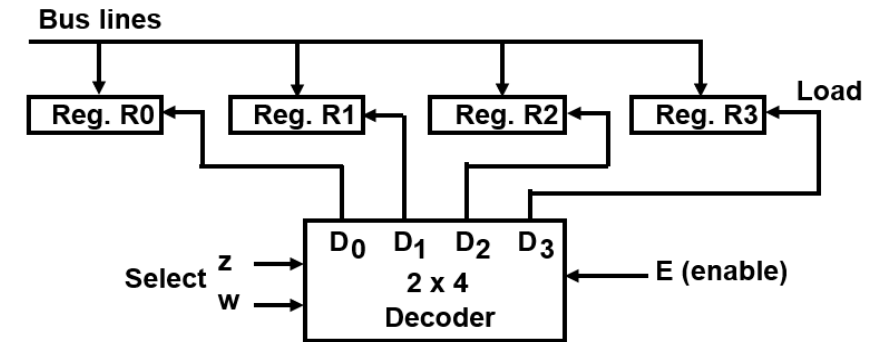
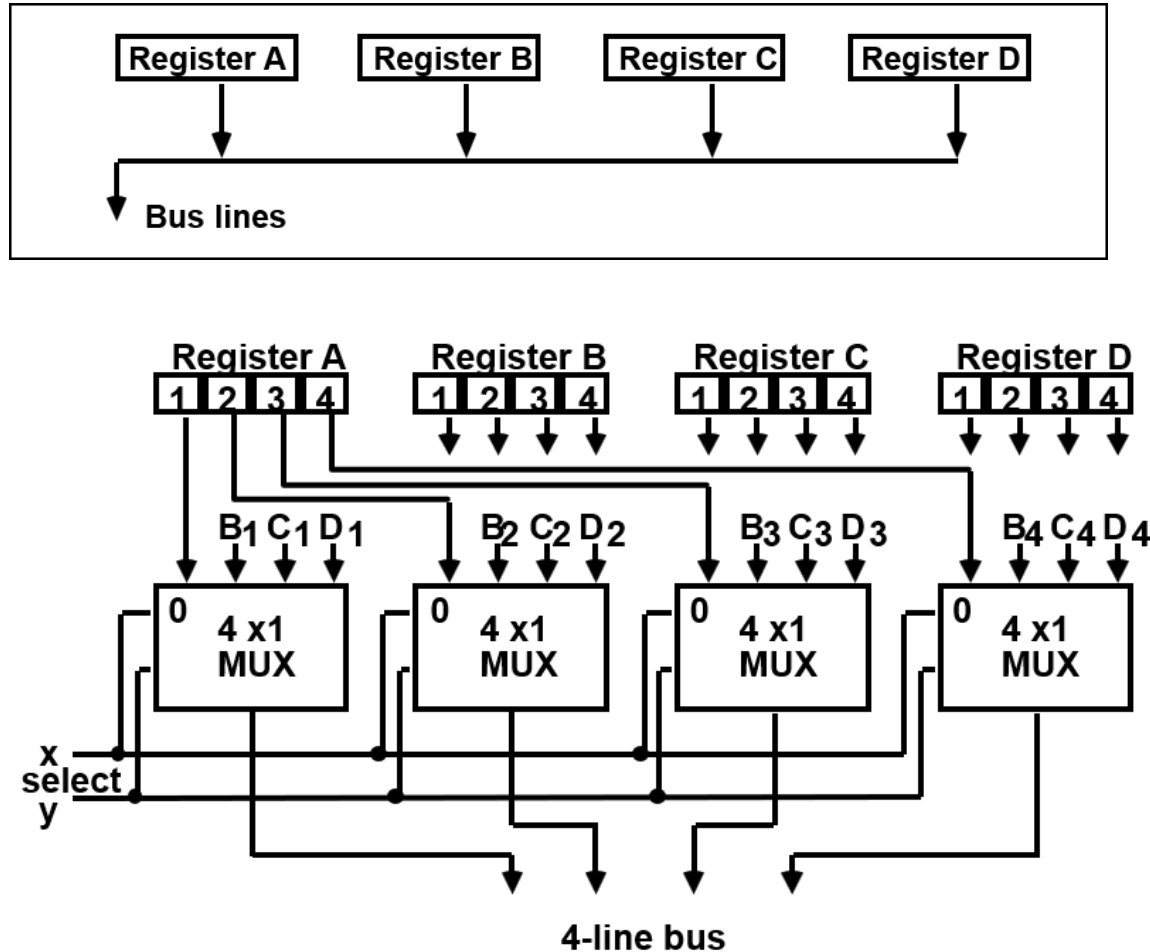
- **BASIC SYMBOLS FOR REGISTER TRANSFERS:**

Capital letters & numerals	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow ←	Denotes transfer of information	R2 ← R1
Colon :	Denotes termination of control function	P:
Comma ,	Separates two micro-operations	A ← B, B ← A

- **CONNECTING REGISTERS:**

- In a digital system with many registers, it is impractical to have data and control lines to directly allow each register to be loaded with the contents of every possible other registers
- To completely connect n registers $\rightarrow n(n-1)$ lines
- $O(n^2)$ cost: This is not a realistic approach to use in a large digital system

- Bus is a path(of a group of wires) over which information is transferred, from any of several sources to any of several destinations.

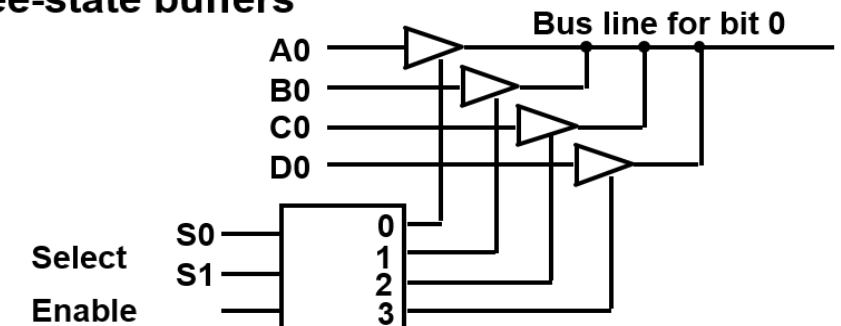


Three-State Bus Buffers

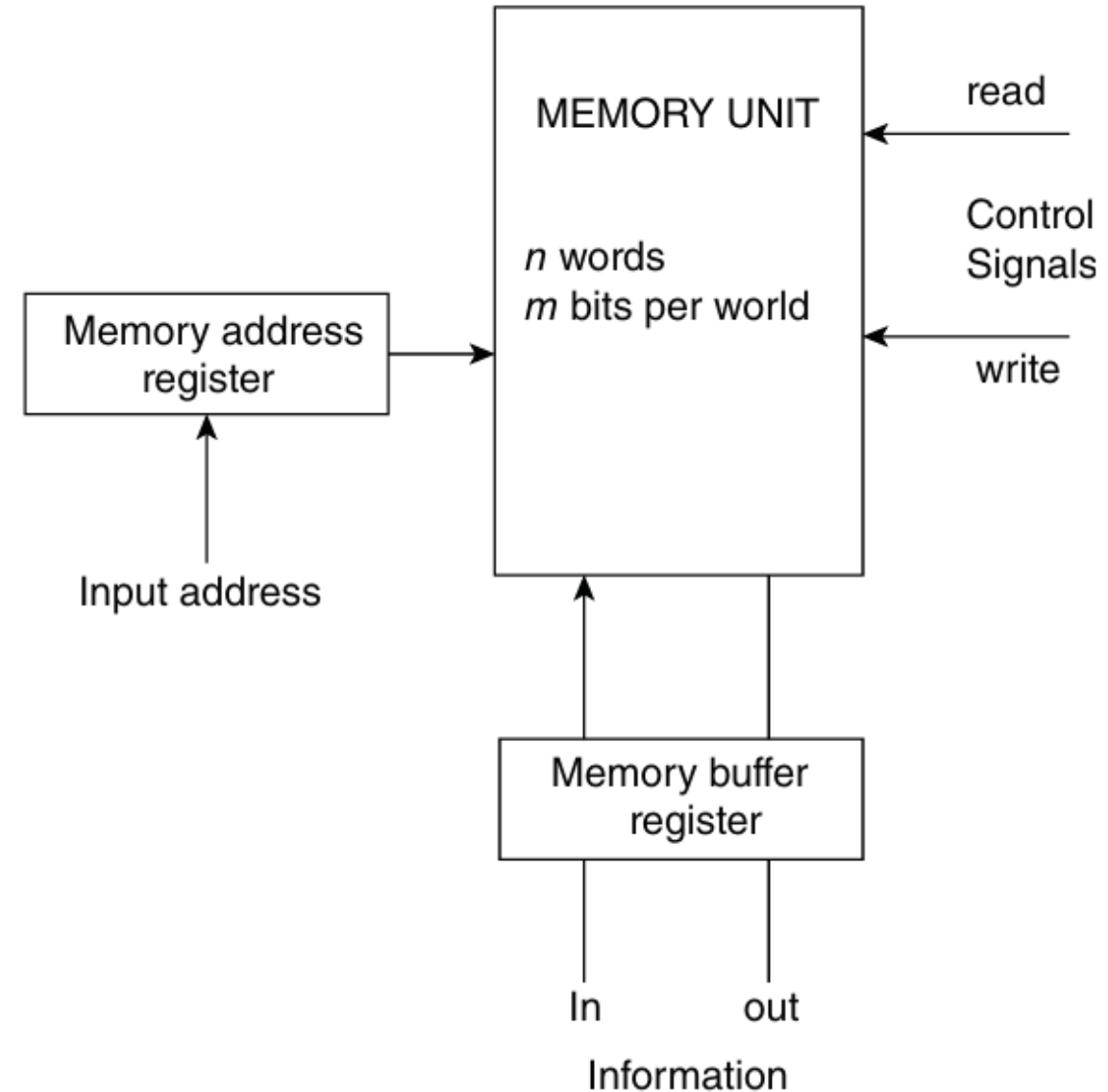
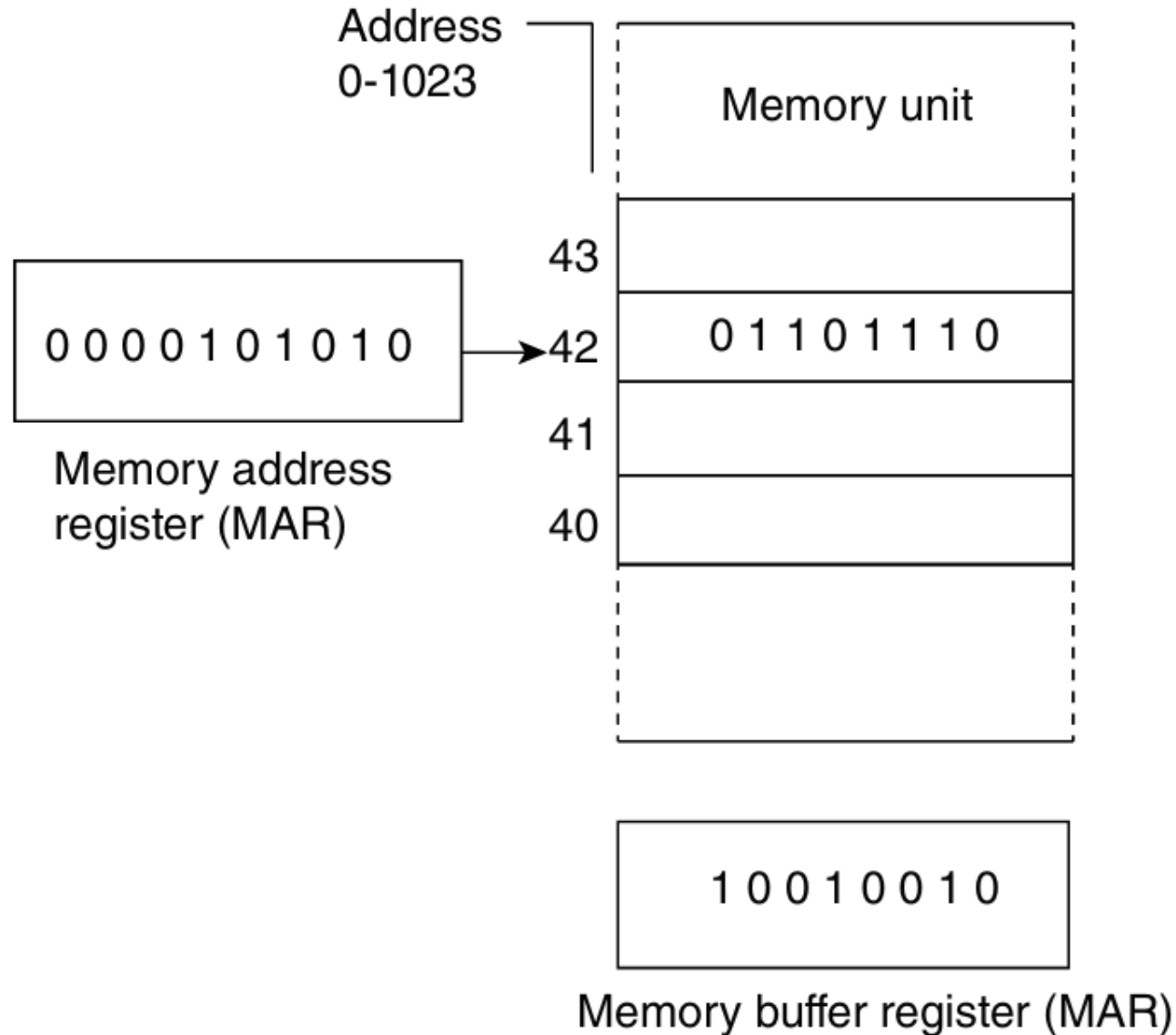
Normal input A
Control input C



Bus line with three-state buffers

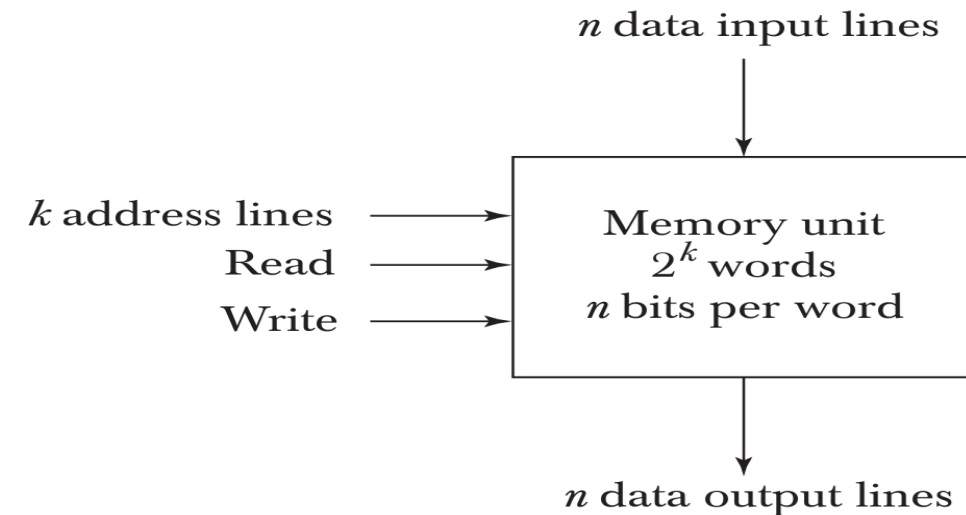
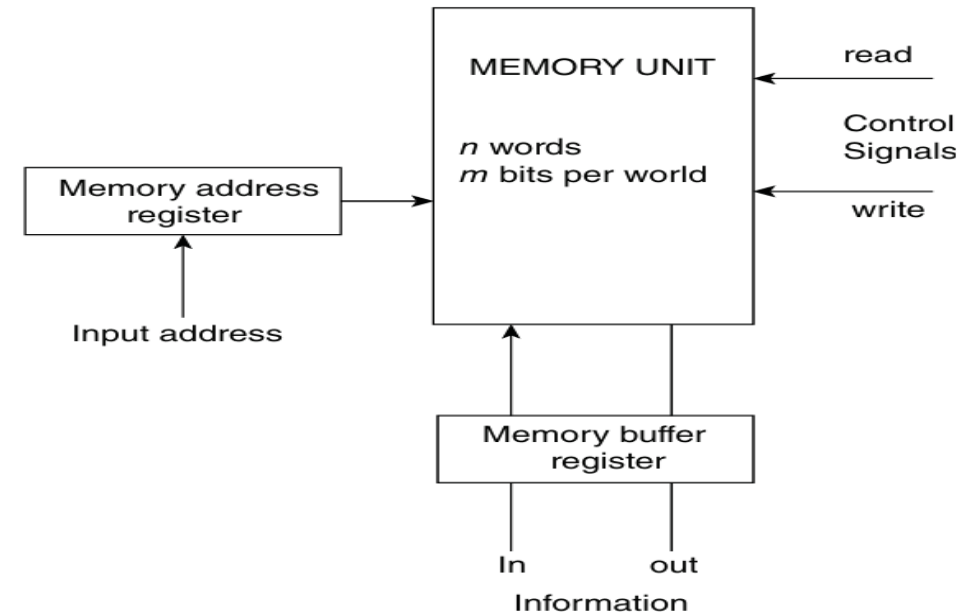


Memory Unit



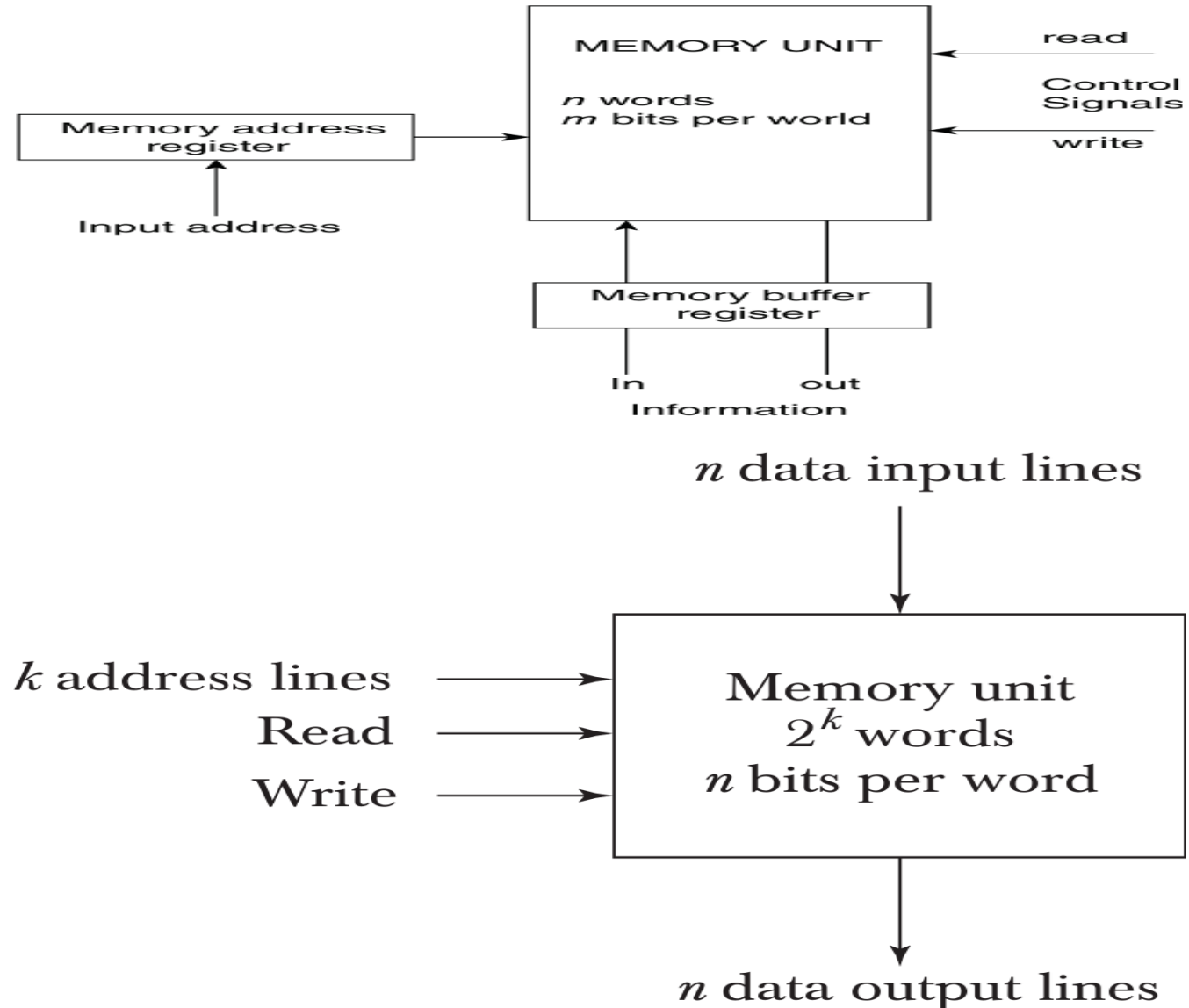
Random Access Memory (RAM)

- An $m \times n$ RAM is an array of binary cells organized into m words of n bits each.
- n = data input/output lines,
- k = address lines provide a binary number of k bits that specify a particular word chosen among the 2^k available inside the memory.



- **RAM – write operation:**

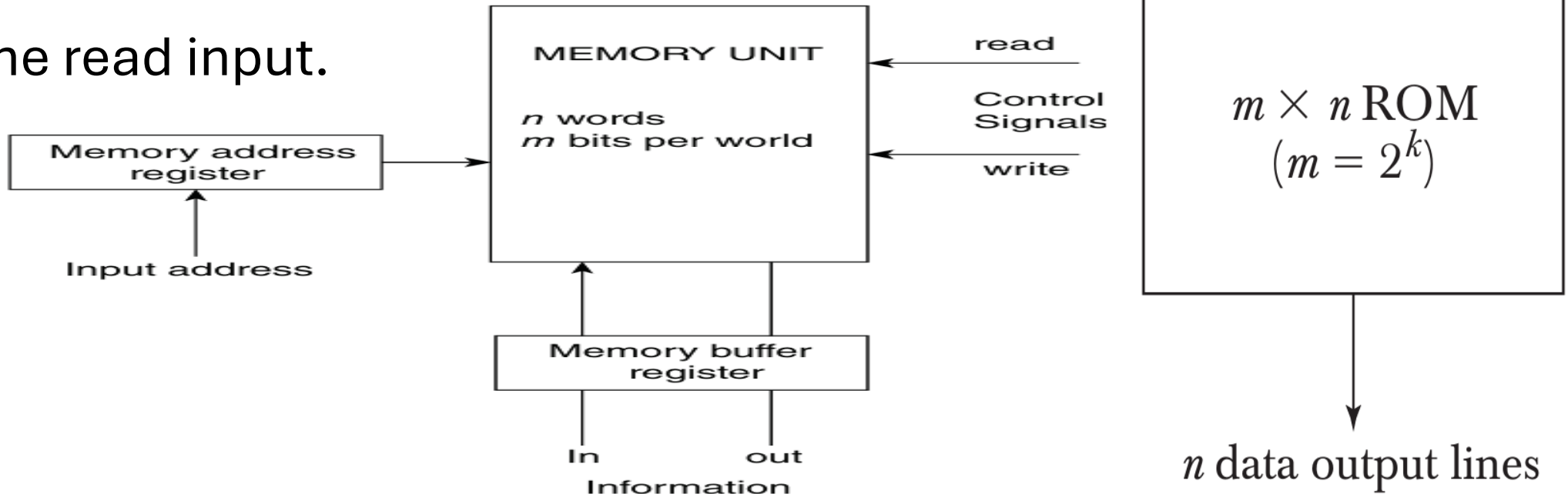
- Apply the binary address into the address lines.
- Apply the data bits into the data input lines.
- Activate the write input.



- An $m \times n$ ROM is an array of binary cells organized into m words of n bits each.

- **ROM – Read operation:**

- Apply the binary address into the address lines.
- Activate the read input.





SUMMARY OF REGISTER & Memory TRANSFER

$A \leftarrow B$	Transfer content of reg. B into reg. A
$AR \leftarrow DR(AD)$	Transfer content of AD portion of reg. DR into reg. AR
$A \leftarrow \text{constant}$	Transfer a binary constant into reg. A
$ABUS \leftarrow R1,$ $R2 \leftarrow ABUS$	Transfer content of R1 into bus A and, at the same time, transfer content of bus A into R2
AR	Address register
DR	Data register
$M[R]$	Memory word specified by reg. R
M	Equivalent to $M[AR]$
$DR \leftarrow M$	Memory <i>read</i> operation: transfers content of memory word specified by AR into DR
$M \leftarrow DR$	Memory <i>write</i> operation: transfers content of DR into memory word specified by AR

Microoperations

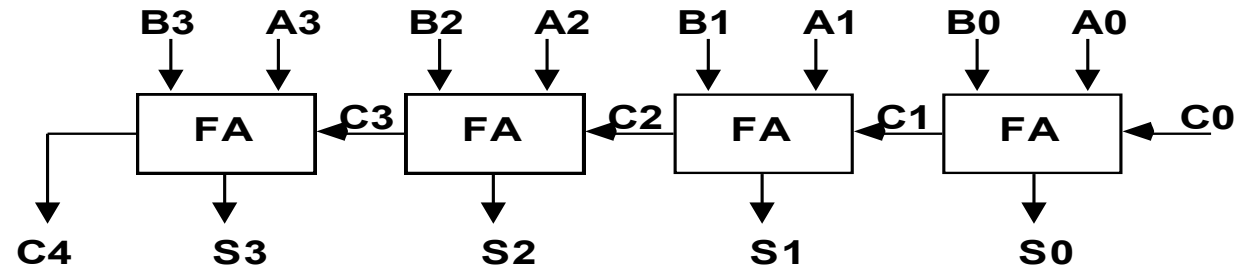
- A **microoperation** is an elementary operation performed with the data.
- The microoperations most often encountered in digital computers are classified into four categories:
 - Register transfer microoperations:** transfer binary information from one register to another.
 - Arithmetic microoperations:** arithmetic operation performed on numeric data stored in registers.
 - Logic microoperations:** bit manipulation operations performed on non-numeric data stored in registers.
 - Shift microoperations:** shift operations performed on data stored in registers.

Arithmetic Microoperations

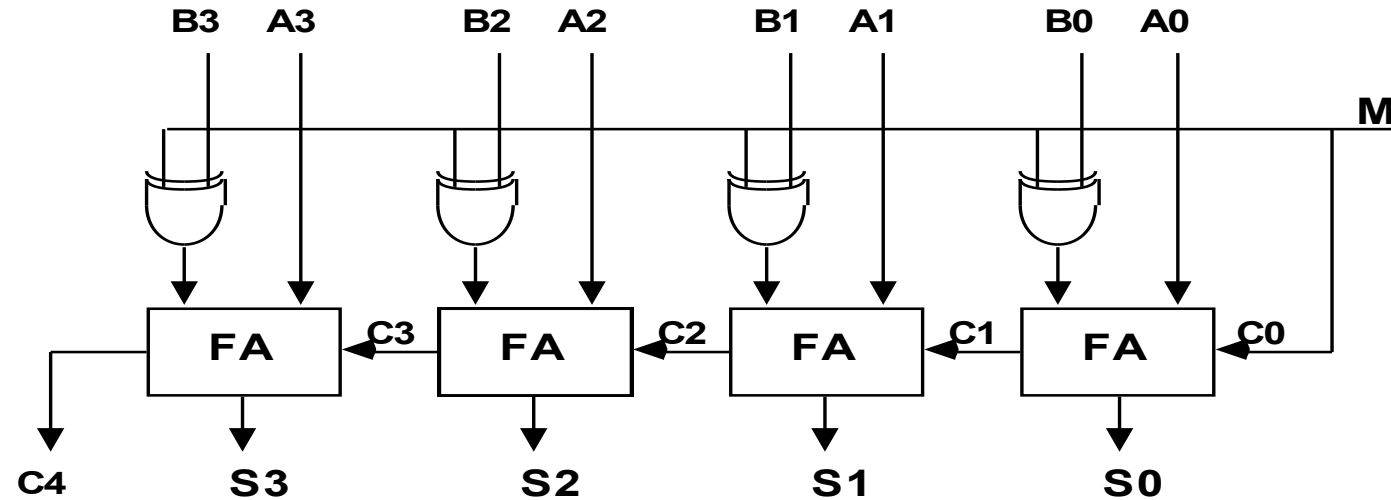
- A **microoperation** is an elementary operation performed with the data.
- **Arithmetic microoperations:** arithmetic operation performed on numeric data stored in registers.

Symbolic designation	Description
$R3 \leftarrow R1 + R2$	Contents of $R1$ plus $R2$ transferred to $R3$
$R3 \leftarrow R1 - R2$	Contents of $R1$ minus $R2$ transferred to $R3$
$R2 \leftarrow \overline{R2}$	Complement the contents of $R2$ (1's complement)
$R2 \leftarrow \overline{R2} + 1$	2's complement the contents of $R2$ (negate)
$R3 \leftarrow R1 + \overline{R2} + 1$	$R1$ plus the 2's complement of $R2$ (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of $R1$ by one
$R1 \leftarrow R1 - 1$	Decrement the contents of $R1$ by one

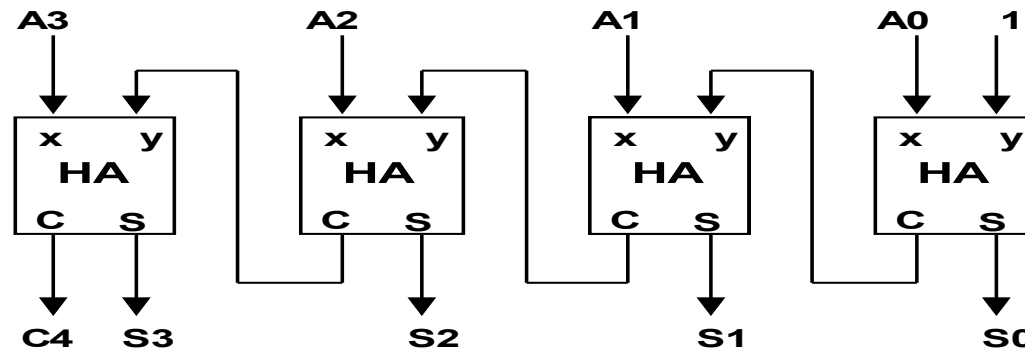
Binary Adder



Binary Adder-Subtractor

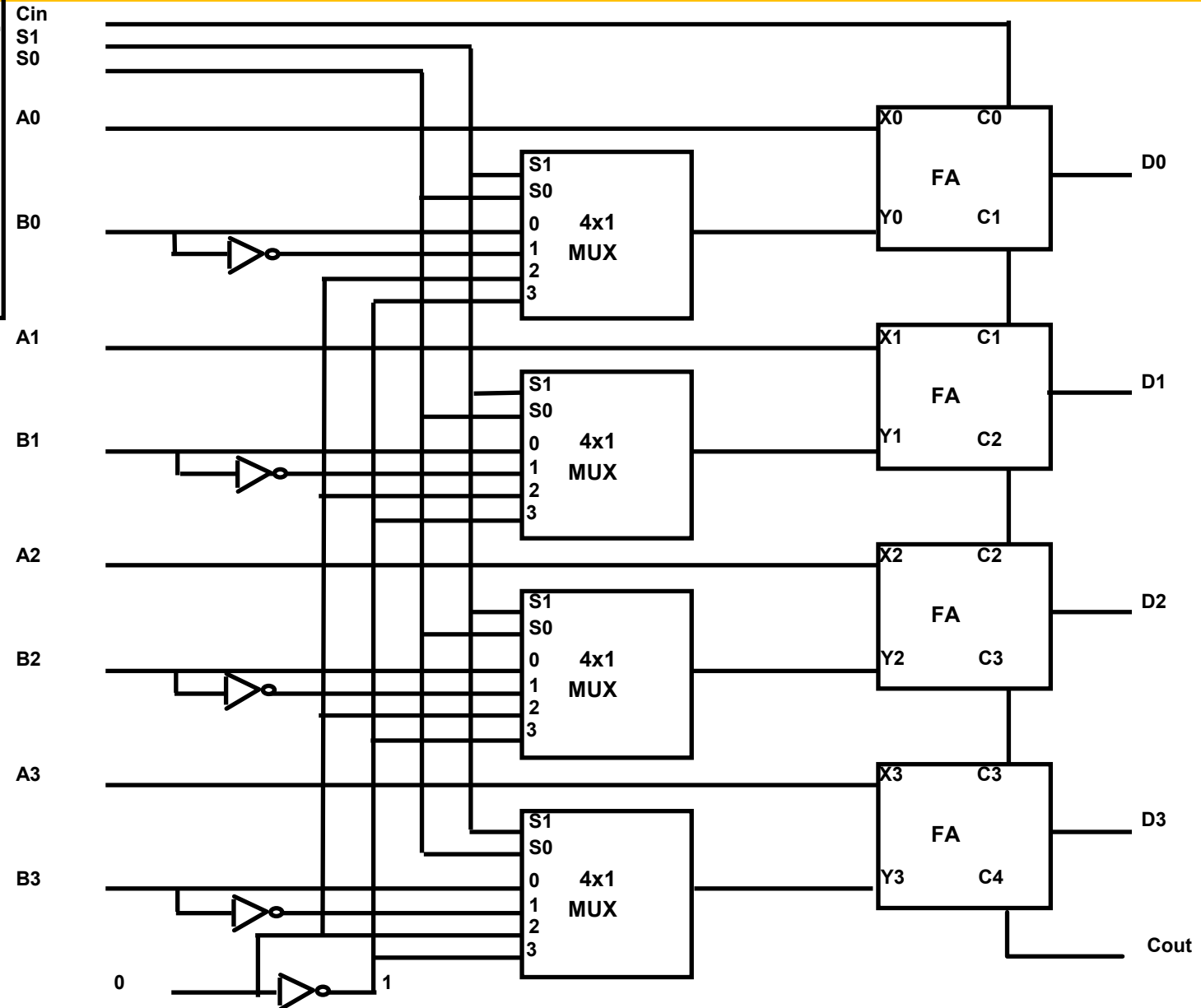


Binary Incrementer



S1	S0	Cin	Y	Output	Microoperation
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	B'	$D = A + B'$	Subtract with borrow
0	1	1	B'	$D = A + B' + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

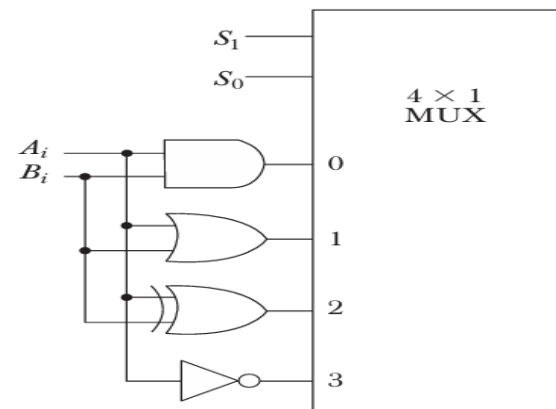
$$D = A + Y + C_{in}$$



- **Logic microoperations:** bit manipulation operations performed on non-numeric data stored in registers.
 - Logic microoperations are bit-wise operations.
 - useful for manipulating individual bits or a portion of binary data.
 - used to change bit values, delete a group of bits, or insert new bit values into a register.
 - useful for making logical decisions based on the bit value

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \bar{B}$	Transfer A
$F_3 = x$	$F \leftarrow A$	
$F_4 = x'y$	$F \leftarrow \bar{A} \wedge B$	Transfer B
$F_5 = y$	$F \leftarrow B$	
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \bar{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \bar{B}$	
$F_{12} = x'$	$F \leftarrow \bar{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \bar{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow A \wedge \bar{B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

Figure 4-10 One stage of logic circuit.



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Functional table

❖ Mask Operation:

- the bits of A are cleared only where there are corresponding 0's in B.
- The **AND microoperation** can be used to **MASK** bits of a register.

- Eg:

1010	<i>A</i> before
1100	<i>B</i> (logic operand)
<u>1000</u>	
1000	<i>A</i> after masking

Selective-set	$A \leftarrow A + B$
Selective-complement	$A \leftarrow A \oplus B$
Selective-clear	$A \leftarrow A \cdot B'$
Mask (Delete)	$A \leftarrow A \cdot B$
Clear	$A \leftarrow A \oplus B$
Insert	$A \leftarrow (A \cdot B) + C$
Compare	$A \leftarrow A \oplus B$

Logic Microoperations

❖ Insert Operation:

- This operation inserts a new value into a group of bits.
- Achieved by
 - first **masking** the bits, it is an AND microoperation and
 - then **insert operation**, by **OR** microoperation with the required value.
- Eg:

Selective-set

$$A \leftarrow A + B$$

Selective-complement

$$A \leftarrow A \oplus B$$

Selective-clear

$$A \leftarrow A \cdot B'$$

Mask (Delete)

$$A \leftarrow A \cdot B$$

Clear

$$A \leftarrow A \oplus B$$

Insert

$$A \leftarrow (A \cdot B) + C$$

Compare

$$A \leftarrow A \oplus B$$

0110 1010

A before

0000 1111

B (mask)

0000 1010

A after masking

0000 1010

A before

1001 0000

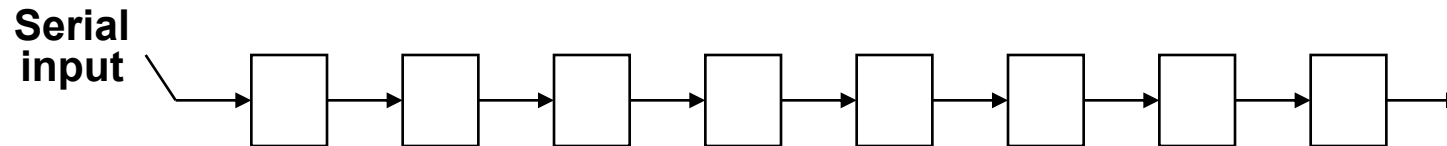
B (insert)

1001 1010

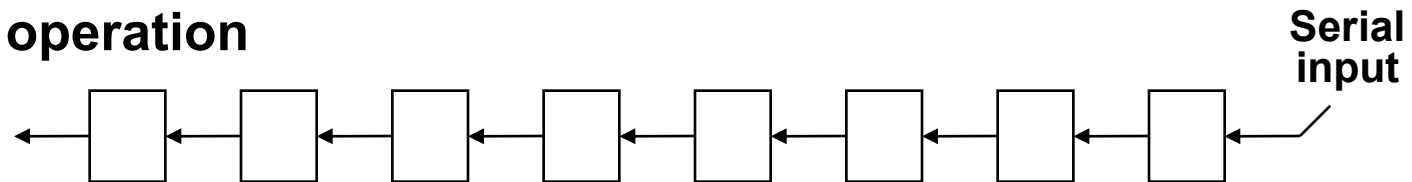
A after insertion

- There are three types of shifts
 - *Logical shift*
 - *Circular shift*
 - *Arithmetic shift*
- What differentiates them is the information that goes into the serial input

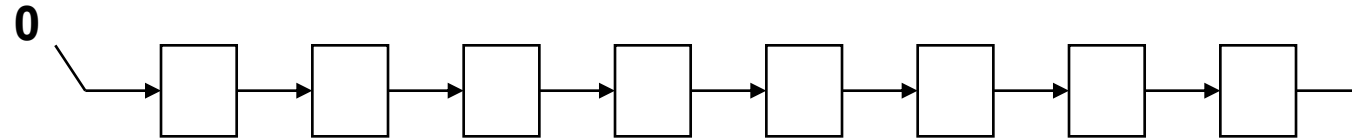
- **A right shift operation**



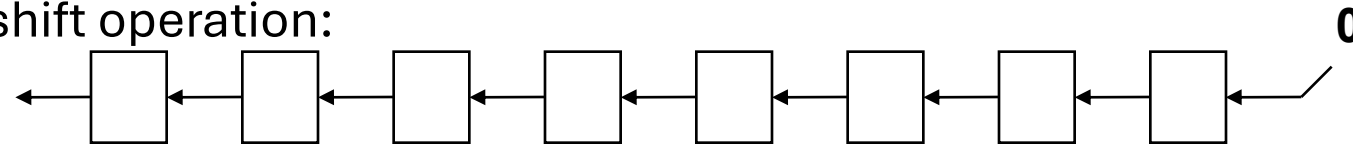
- **A left shift operation**



- In a logical shift the serial input to the shift is a 0.
- A right logical shift operation:



- A left logical shift operation:

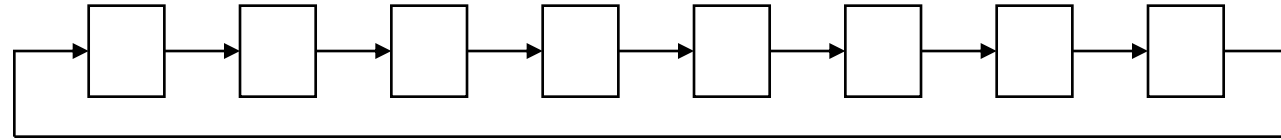


- In a Register Transfer Language, the following notation is used
 - *shl* for a logical shift left
 - *shr* for a logical shift right
 - Examples:
 - $R2 \leftarrow shr\ R2$
 - $R3 \leftarrow shl\ R3$

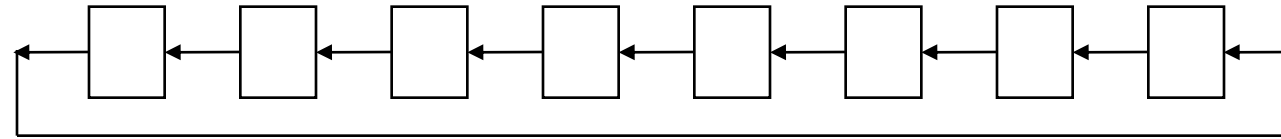
CIRCULAR SHIFT

- also known as a **Rotate** operation
- In a circular shift the serial input is the bit that is shifted out of the other end of the register.

- A right circular shift operation:



- A left circular shift operation:

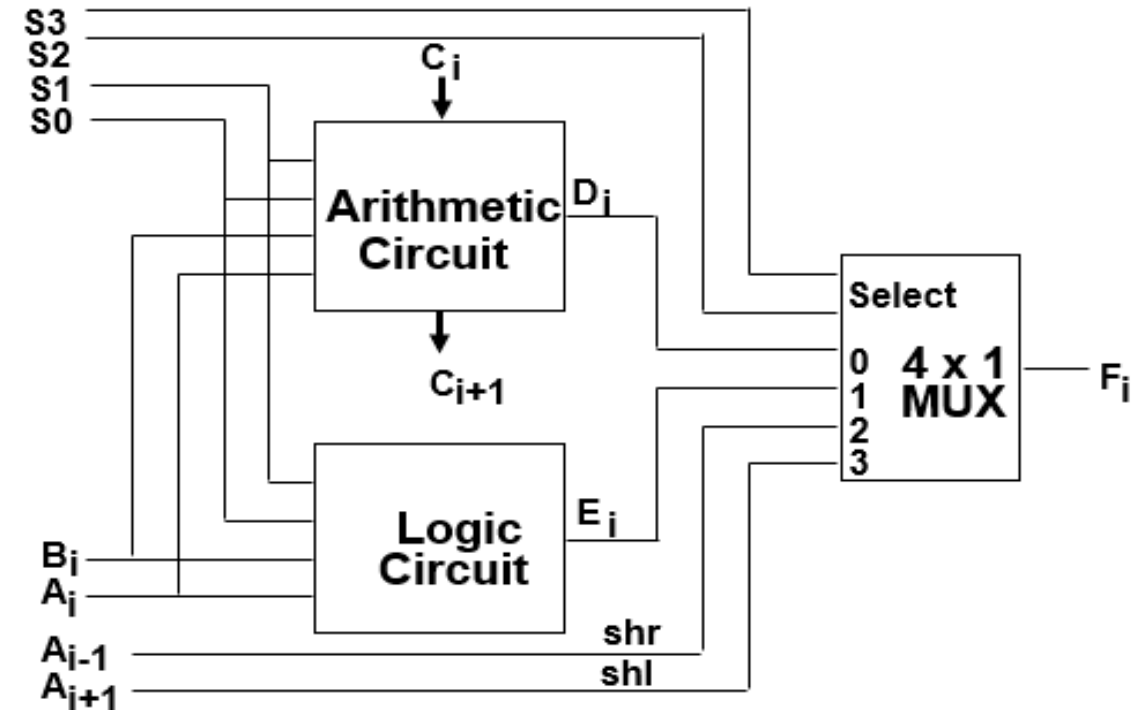


- In a RTL, the following notation is used
 - *cil* for a circular shift left
 - *cir* for a circular shift right
 - Examples:
 - $R2 \leftarrow cir R2$
 - $R3 \leftarrow cil R3$

QnA

Design a 4-bit ALU. That may perform the following operation.

Function
Transfer A
Increment A
Addition
Add with carry
Subtract with borrow
Subtraction
Decrement A
Transfer A
AND
OR
XOR
Complement A
Shift right A into F
Shift left A into F



S3	S2	S1	S0	Cin	Operation	Function
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + B'$	Subtract with borrow
0	0	1	0	1	$F = A + B' + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	X	$F = A \wedge B$	AND
0	1	0	1	X	$F = A \vee B$	OR
0	1	1	0	X	$F = A \oplus B$	XOR
0	1	1	1	X	$F = A'$	Complement A
1	0	X	X	X	$F = \text{shr } A$	Shift right A into F
1	1	X	X	X	$F = \text{shl } A$	Shift left A into F

Explain its working in detail.

- 1) **M. Morris Mano, “Computer System Architecture”, Pearson, (3e), 2007.**
- 2) **M. Morris Mano - PPT**